

Diseñar e implementar un editor estilo Lopaka como un componente autocontenido en React y TypeScript

Resumen ejecutivo

La documentación oficial de Lopaka describe un editor con una rejilla de cinco regiones —panel de capas, barra superior, canvas, inspector y panel de código/configuración— y un conjunto de interacciones muy concretas: selección, marquee selection, mover, redimensionar con modificadores, dibujo pixel a pixel, inserción de imágenes y generación de código en tiempo real. El repositorio también deja ver una arquitectura interna basada en herramientas y plugins: un registro de `tools`, una cadena de plugins para `paint`, `add`, `resize`, `select`, `move`, `copy`, `history`, `delete`, `image-drop`, `group` y `zoom`, y un enrutador de eventos que centraliza mouse, teclado, drag-and-drop y touch. En React + TypeScript, la réplica más sólida no es “dibujar cosas sueltas sobre un canvas”, sino construir un **modelo de documento propio, tipado, serializable y desacoplado del renderer**, gobernado por `useReducer`, con estado derivado mínimo y con controladores de herramienta estilo plugin. ¹

Mi recomendación principal es un **renderer híbrido: SVG** para geometría vectorial, texto, handles, vértices, marquee y overlays de selección; **canvas u OffscreenCanvas** para bitmaps y pintura libre. La razón es técnica y no estética: SVG ya define primitivas nativas para `rect`, `circle`, `ellipse`, `line`, `polyline`, `polygon` y `text`, mientras que el canvas 2D es una superficie bitmap con origen en la esquina superior izquierda, ideal para edición pixel a pixel. Konva y Fabric resuelven bien la interacción sobre canvas, pero ambos introducen desacoples relevantes si se toman como fuente de verdad del estado: en Konva, `Transformer` modifica `scaleX/scaleY` en vez de `width/height`, y su serialización JSON no incluye handlers ni imágenes; en Fabric, la serialización restaura estado visual, pero no los controles o handlers personalizados. Para un inspector numérico, una historia undo/redo precisa y una generación de código estable, conviene que el **modelo de dominio sea propio** y que el renderer sea solo un adaptador. ²

El punto donde más crece la complejidad no es dibujar rectángulos, sino reproducir con fidelidad las **semánticas de interacción**: texto centrado al crear, pintura con borrado por clic derecho, líneas con endpoints editables, triángulos y polígonos con edición de vértices, y un history stack que agrupe gestos completos en una sola operación de undo/redo. Eso es exactamente donde la combinación de reducir + comandos inversos + eventos pointer unificados + refs para estado transitorio da el mejor equilibrio entre robustez y mantenibilidad. ³

Alcance funcional y criterio de paridad

Lopaka documenta una top bar con selección, herramientas de dibujo soportadas por la plataforma activa y zoom; un canvas con selección, rubber-band, drag, resize, zoom y pan; un inspector con propiedades por

tipo de capa; y un panel de código que se regenera en tiempo real, puede resaltar la línea del objeto seleccionado y soporta prefijo/sufijo editables alrededor del bloque generado. También documenta soportes específicos para imagen, paint, texto, rectángulo, círculo, elipse, línea, triángulo, polígono y, en LVGL, widgets adicionales. 4

Para el componente que me pediste, propongo una **paridad focalizada**: `select`, `image`, `paint`, `text`, `rectangle`, `circle`, `line`, `triangle` y `polygon`, con **una sola screen** y **sin navegación entre screens**. Lopaka sí soporta múltiples screens, reordenamiento y menú contextual sobre ellas, pero esa capacidad debe quedar **deliberadamente fuera del alcance** de este componente; a lo sumo, conviene mantener en el layout una zona izquierda compatible con una futura ampliación, mostrando una sola entrada fija y la lista de layers. La elipse y los widgets LVGL deben quedar como extensiones feature-flagged, no como deuda de la primera versión. 5

Arquitectura propuesta

El punto de entrada debe ser **una sola exportación pública**, por ejemplo `LopakaLikeEditor`, que internamente usa subcomponentes privados y hooks locales. En términos de React, esto encaja bien con una UI compuesta por piezas, con estado elevado al ancestro común, y con `reducer/context` cuando la pantalla ya es lo bastante compleja como para que pasar estado y `dispatch` por props se vuelva frágil. Los refs deben reservarse para cosas no render-dependientes: `pointer capture`, sesión de drag, `canvas offscreen`, `scroll sincronizado` y edición temporal de texto. 6

```
graph TD
  A[LopakaLikeEditor] --> B[Toolbar]
  A --> C[Workspace]
  A --> D[CodePanel]
  A --> E[ImagePanelModal]

  C --> F[LeftSidebar]
  C --> G[CanvasViewport]
  C --> H[PropertiesPanel]

  F --> I[SingleScreenStub]
  F --> J[LayersList]

  G --> K[BitmapStage Canvas]
  G --> L[VectorStage SVG]
  G --> M[SelectionOverlay]
  G --> N[Rulers and Marquee]

  D --> O[GeneratedCodeView]
  D --> P[CodeSettings]

  E --> Q[UploadSource]
```

```
E --> R[AssetLibrary]
E --> S[BitmapTransformEditor]
```

La arquitectura interna que mejor replica el comportamiento documentado de Lopaka, pero sin amarrarse a su stack original en Vue, es esta:

Interno	Responsabilidad principal	Fuente de verdad
<code>LopakaLikeEditor</code>	Orquestación, reducir, history, callbacks externos	<code>EditorState</code>
<code>Toolbar</code>	Activar herramienta, zoom, color mode, color de pincel	<code>ui.tool</code> , <code>viewport</code> , <code>paint</code>
<code>LeftSidebar</code>	Screen única, order de layers, lock/hidden, z-order	<code>document.zOrder</code> , <code>objectsById</code>
<code>CanvasViewport</code>	Coordenadas, pointer routing, drag sessions, pan/zoom	<code>viewport</code> + refs transitorios
<code>BitmapStage</code>	Render de imágenes y capas de paint	<code>objectsById</code> rasterables
<code>VectorStage</code>	Render de rect/circle/line/triangle/polygon/text	<code>objectsById</code> vectoriales
<code>SelectionOverlay</code>	Handles, vértices, marquee, hover target	<code>selection</code> , <code>interaction</code>
<code>PropertiesPanel</code>	Edición precisa numérica y por tipo	<code>selection</code> + <code>objectsById</code>
<code>CodePanel</code>	IR + texto generado + lineMap por objeto	selectores derivados
<code>ImagePanelModal</code>	Upload, asset reuse, preview, crop/resize básico	<code>assetsById</code>

La firma pública del componente puede mantenerse pequeña, incluso si la implementación interna crece bastante:

```
export interface LopakaEditorProps {
  initialDocument?: EditorDocument;
  platformPreset?: 'generic-json' | 'u8g2' | 'adafruit-gfx' | 'tft-espi';
  readOnly?: boolean;
  onDocumentChange?: (doc: EditorDocument) => void;
  onCodeChange?: (result: GeneratedCodeResult) => void;
  onDirtyChange?: (dirty: boolean) => void;
}

export function LopakaLikeEditor(
  props: LopakaEditorProps
): React.JSX.Element {
```

```
// implementación
}
```

Si la exigencia de “single self-contained component” significa **una sola unidad pública reutilizable**, esta forma es ideal. Si significa **un único archivo físico**, aún puede cumplirse moviendo hooks y subcomponentes privados al mismo archivo, pero eso lo recomiendo solo para prototipo o demo; para producción, conviene un único export público y módulos internos co-localizados.

Modelo de estado y eventos

React recomienda agrupar estado que cambia junto, evitar redundancia y duplicación, y preferir estructuras planas sobre árboles profundamente anidados. Ese consejo encaja perfectamente con un editor visual: el documento debe estar **normalizado**, la selección debe guardar **IDs y no objetos enteros**, y el código generado debe ser **estado derivado**, no persistido como string mutable en el store. TypeScript, por su parte, resuelve muy bien este problema con interfaces para contratos estables y uniones discriminadas para variantes por herramienta y tipo de objeto. ⁷

```
export type ToolId =
  | 'select'
  | 'image'
  | 'paint'
  | 'text'
  | 'rectangle'
  | 'circle'
  | 'line'
  | 'triangle'
  | 'polygon';

export interface ScreenSpec {
  width: number;
  height: number;
  background: string;
  title: string;
}

export interface BaseObject {
  id: string;
  kind: CanvasObject['kind'];
  name: string;
  locked: boolean;
  hidden: boolean;
  overlay?: boolean;
  zIndex: number;
}

export interface RectangleObject extends BaseObject {
```

```

    kind: 'rectangle';
    x: number;
    y: number;
    width: number;
    height: number;
    fill: string | null;
    stroke: string | null;
    strokeWidth: number;
}

export interface CircleObject extends BaseObject {
    kind: 'circle';
    cx: number;
    cy: number;
    radius: number;
    fill: string | null;
    stroke: string | null;
    strokeWidth: number;
}

export interface LineObject extends BaseObject {
    kind: 'line';
    x1: number;
    y1: number;
    x2: number;
    y2: number;
    stroke: string;
    strokeWidth: number;
}

export interface TriangleObject extends BaseObject {
    kind: 'triangle';
    points: [{ x: number; y: number }, { x: number; y: number }, { x: number; y:
number }];
    fill: string | null;
    stroke: string | null;
    strokeWidth: number;
}

export interface PolygonObject extends BaseObject {
    kind: 'polygon';
    points: Array<{ x: number; y: number }>;
    fill: string | null;
    stroke: string | null;
    strokeWidth: number;
}

export interface TextObject extends BaseObject {

```

```

    kind: 'text';
    x: number;
    y: number;
    text: string;
    fontFamily: string;
    fontSize: number;
    fill: string;
    align?: 'left' | 'center' | 'right';
}

export interface ImageObject extends BaseObject {
    kind: 'image';
    x: number;
    y: number;
    width: number;
    height: number;
    assetId: string;
    alphaChannel?: boolean;
}

export interface BitmapObject extends BaseObject {
    kind: 'bitmap';
    x: number;
    y: number;
    width: number;
    height: number;
    pixelData: Uint8ClampedArray; // o un handle a OffscreenCanvas
    colorMode: 'monochrome' | 'rgb';
    alphaChannel?: boolean;
}

export type CanvasObject =
    | RectangleObject
    | CircleObject
    | LineObject
    | TriangleObject
    | PolygonObject
    | TextObject
    | ImageObject
    | BitmapObject;

export interface EditorDocument {
    screen: ScreenSpec;
    objectsById: Record<string, CanvasObject>;
    zOrder: string[];
    assetsById: Record<string, ImageAsset>;
}

```

```

export interface SelectionState {
  selectedIds: string[];
  vertexEdit?:
    | { objectId: string; kind: 'triangle'; activeVertex?: 0 | 1 | 2 }
    | { objectId: string; kind: 'polygon'; activeVertex?: number };
}

export interface HistoryEntry {
  label: string;
  forward: EditorAction[];
  inverse: EditorAction[];
  timestamp: number;
}

export interface EditorState {
  document: EditorDocument;
  selection: SelectionState;
  activeTool: ToolId | null;
  hoveredId: string | null;
  viewport: { zoom: number; panX: number; panY: number };
  history: {
    past: HistoryEntry[];
    future: HistoryEntry[];
    transactionLabel?: string;
    stagedForward: EditorAction[];
    stagedInverse: EditorAction[];
  };
  ui: {
    rightMouseMode: 'none' | 'erase' | 'remove-vertex';
    textEditingId?: string;
    imagePanelOpen: boolean;
    codeSettings: CodegenSettings;
  };
}

```

También conviene definir un conjunto de acciones muy explícito. El patrón de TypeScript que mejor funciona aquí es: **interfaces para los contratos estables** y **type unions discriminadas** para acciones y objetos. Eso permite `switch(action.type)` con narrowing seguro y evita una cantidad enorme de errores sutiles en herramientas complejas. ⁸

```

export type EditorAction =
  | { type: 'tool/set'; tool: ToolId | null }
  | { type: 'selection/set'; ids: string[] }
  | { type: 'selection/vertex-mode'; value?: SelectionState['vertexEdit'] }
  | { type: 'object/create'; object: CanvasObject }
  | { type: 'object/update'; id: string; patch: Partial<CanvasObject> }

```

```
| { type: 'object/delete'; ids: string[] }
| { type: 'object/reorder'; zOrder: string[] }
| { type: 'asset/register'; asset: ImageAsset }
| { type: 'viewport/set'; zoom?: number; panX?: number; panY?: number }
| { type: 'history/begin'; label: string }
| { type: 'history/commit' }
| { type: 'history/undo' }
| { type: 'history/redo' };
```

Para undo/redo, no recomiendo snapshots completos como estrategia principal, salvo prototipo. El repositorio de Lopaka ya evidencia que la historia real necesita **lotes**, pares undo/redo e incluso marcadores de batch para agrupar gestos. En tu componente, el equivalente correcto es un **command stack con acciones inversas**, donde cada gesto de usuario se inicia en `pointerdown` o `history/begin` y se confirma en `pointerup`, `dblclick`, `Escape` o “Apply” del panel. Eso hace que “arrastrar una figura”, “mover tres vértices” o “editar varias propiedades numéricas seguidas” se comporten como una única operación de usuario. ⁹

El modelo de eventos debe usar **Pointer Events como eje central**. La especificación W3C define `pointerdown`, `pointermove` y `pointerup` como un marco unificado para mouse, touch y stylus, y React expone además `onContextMenu`, `onDoubleClick`, `onKeyDown`, `onKeyUp`, `onDragOver` y `onDrop`. En la práctica, eso significa: canvas con `tabIndex={0}` para recibir teclado; enrutamiento de todo pointer input por un `CanvasViewport`; supresión selectiva del menú contextual cuando el clic derecho significa “erase pixel” o “remove vertex”; y `preventDefault()` en `onDragOver` para permitir drop de imágenes. ¹⁰

Los estados transitorios de interacción —por ejemplo, punto inicial de drag, bbox original, handle activo, trayectoria del pincel o si el pointer está capturado— deben vivir en `useRef`, no en React state. React documenta precisamente que los refs persisten entre renders y no disparan re-render al actualizarse, lo que los hace apropiados para interacción de alta frecuencia y para manipulación directa del DOM cuando hace falta medir, enfocar o capturar. ¹¹

Herramientas, renderizado y paneles

Elección del renderer

La comparación práctica entre SVG, canvas puro e híbrido se puede resumir así:

Opción	Ventajas	Costes
SVG puro	DOM nativo, texto y shapes declarativos, hit-testing y focus más simples, handles fáciles	Pintura bitmap/pixel costosa, demasiados nodos si crece el documento
Canvas puro	Excelente para bitmaps y muchas entidades, superficie continua, export sencillo	Hit-testing manual, accesibilidad más difícil, inspector/handles más costosos
Híbrido	Vector + texto accesibles en SVG, bitmaps/paint eficientes en canvas, overlays limpios	Dos pipelines de render y coordinación de z-order / eventos

La base objetiva de esa tabla es clara: SVG está diseñado para gráficos vectoriales y mixtos vector/raster, y define primitivas nativas para las formas que este editor necesita; el canvas 2D es una superficie bitmap cartesiana ideal para pintar píxel a píxel; Konva y Fabric añaden un modelo de objetos, eventos y serialización sobre canvas, pero con semánticas que conviene tratar como adaptadores y no como fuente de verdad. ¹²

Mi recomendación concreta es:

- `BitmapStage` en `<canvas>` o `OffscreenCanvas` para `bitmap` e `image`.
- `VectorStage` en `<svg>` para `rectangle`, `circle`, `line`, `triangle`, `polygon`, `text`.
- `SelectionOverlay` también en `<svg>` para handles y vértices.
- Rasters representados en el stage superior como `<image>` enlazadas a blobs/data URLs, o, si prefieres menos capas visuales, como un canvas base único.

Si el objetivo fuese acelerar implementación a costa de menor control semántico, `react-konva` es la opción más alineada con React porque tiene bindings oficiales y un sistema completo de shapes, eventos, drag y Transformer. Aun así, yo mantendría el **document model propio** y usaría Konva solo como backend visual opcional, precisamente por las diferencias entre transform visual y propiedades geométricas persistentes. ¹³

Implementación por herramienta

La documentación de Lopaka deja bastantes comportamientos ya fijados: `Escape` vuelve a select, `Text` inserta al centro, `Paint` borra con clic derecho y permite líneas rectas con `Shift`, `Triangle` se crea por múltiples clicks, `Polygon` usa multi-click + doble click + edición de vértices, y el inspector expone coordenadas, tamaños, colores, textos, alpha y operaciones de imagen. El repositorio confirma además que `TextTool` centra el texto al activarse, `ImageTool` abre un wizard, `PaintTool` reutiliza una capa bitmap seleccionada si existe, y `LineLayer` trabaja naturalmente con `x1/y1/x2/y2` y snapping por `Shift`. ¹⁴

Herramienta	Creación	Edición posterior	Clic derecho / doble click	Mapeo a código
Select / Pointer	Sin creación	Selección, marquee, move, resize, multi-select	Doble click entra en modos especiales según objeto	No genera draw op
Image	Abre panel/modal de assets o upload	Move, resize, alpha, flip/rotate/invert	Doble click puede reabrir edición avanzada	<code>bitmap/</code> <code>image</code>
Paint / Pencil	Crea o reutiliza capa bitmap	Pintar, borrar, transformar bitmap	Clic derecho = erase; <code>Shift</code> = trazo recto	<code>bitmap</code> o array raster
Text	Inserta centrado y enfoca editor	Move, resize bbox, editar texto/fuente	Doble click = edición in-place	<code>text</code>

Herramienta	Creación	Edición posterior	Clic derecho / doble click	Mapeo a código
Rectangle	Click-drag bbox	Move, resize, fill/stroke	—	<code>rect</code>
Circle	Click-drag desde centro o bbox normalizado	Move, resize, radius	—	<code>circle</code>
Line	Click-drag start/end	Mover línea completa o endpoints	—	<code>line</code>
Triangle	Tres clicks o preview con clicks secuenciales	Vértices editables, move bbox	Doble click = vertex mode	<code>polygon</code> de 3 puntos
Polygon	Multi-click con preview	Vértices editables, move bbox	Doble click = vertex mode; clic derecho = remove vertex	<code>polygon</code>

Para la herramienta de línea, yo adoptaría de entrada el modelo explícito `x1/y1/x2/y2`, porque coincide con el inspector documentado y con la propia `LineLayer` del repositorio. Además, añadiría snapping con `Shift` a horizontal, vertical y 45°, porque ya está implementado en la capa de Lopaka original y encaja muy bien en un inspector geométrico. ¹⁵

Para triángulos y polígonos, la recomendación es **no simplificarlos a bbox + rotate**. Lopaka ya documenta multi-click y edición de vértices, y su release 0.6 añade marcadores circulares, flujo con `Escape`, remoción de vértices con clic derecho y handles más grandes para facilidad de uso. La réplica fiel, por tanto, es: un modo de creación, un modo de edición de vértices y un modo normal de selección/movimiento. ¹⁶

Hit-testing, handles y edición precisa

Para formas vectoriales en SVG, aprovéchate del hit-test nativo del DOM cuando el usuario haga click sobre la propia shape. El problema real aparece con líneas finas y vértices pequeños: ahí conviene **inflar la región interactiva** de forma lógica, aunque el trazo visible siga siendo delgado. Konva documenta exactamente esta necesidad con `hitStrokeWidth` y `hitFunc`; Lopaka, en paralelo, ha terminado agrandando sus handles y marcadores de vértices por motivos de precisión y accesibilidad. Esa es la misma decisión correcta aquí: **make hit areas bigger than visuals**. ¹⁷

Mi propuesta concreta de handles es:

- Objetos con bbox: 8 handles rectangulares o circulares.
- Línea: 2 handles de endpoint más una zona central para mover toda la línea.
- Triángulo: 3 handles de vértice y un bbox opcional para move/scale global.
- Polígono: N handles de vértice; click-drag mueve un vértice; click interior mueve la forma completa; clic derecho sobre handle elimina el vértice si el polígono sigue siendo válido.

El inspector debe ser el mecanismo de edición **más preciso**; el drag debe ser el mecanismo **más rápido**. Esa división también coincide con la UI documentada de Lopaka, donde el panel derecho cambia por tipo de layer y ofrece coordenadas, tamaños, radios, endpoints, fill, texto, color, alpha, actions y delete. ¹⁸

Una API limpia para el panel de propiedades se puede ver así:

```
export type SharedPatch =
  | { x?: number; y?: number; width?: number; height?: number }
  | { hidden?: boolean; locked?: boolean; overlay?: boolean };

export interface PropertiesPanelProps {
  selected: CanvasObject[];
  screen: ScreenSpec;
  onPatchObject: (id: string, patch: Partial<CanvasObject>) => void;
  onPatchMany: (ids: string[], patch: SharedPatch) => void;
  onDelete: (ids: string[]) => void;
  onAlign: (
    mode:
      | 'left'
      | 'center-x'
      | 'right'
      | 'top'
      | 'center-y'
      | 'bottom'
  ) => void;
}
```

En el contenido del inspector, yo distinguiría tres niveles:

1. **Común a casi todo:** `x`, `y`, `width`, `height`, `hidden`, `locked`, `overlay`, `zIndex`.
2. **Geométrico por tipo:** `x1/y1/x2/y2` para línea; `radius` para círculo; `points[]` para triángulo/polígono.
3. **Semántico/visual:** `stroke`, `fill`, `strokeWidth`, `fontFamily`, `fontSize`, `text`, `assetId`, `alphaChannel`, `colorMode`.

Eso refleja bastante bien los campos que Lopaka documenta en su inspector: `X`, `Y`, `W`, `H`, `X1`, `Y1`, `X2`, `Y2`, `Radius/RX/Ry`, `Fill`, `Inverted`, `Color`, `Font`, `Text`, `Overlay`, `Alpha channel` y acciones de imagen. ¹⁸

Panel de imágenes y flujo bitmap

Lopaka documenta tres entradas al flujo de imagen: icon packs, upload de raster images/GIF y entrada de byte array; además permite saltar el wizard soltando PNG/BMP directamente al canvas. Tras importarse, la imagen se convierte en una `Paint` /bitmap layer, editable con el Paint tool; en el inspector aparecen transformaciones como flip horizontal, flip vertical, rotate, invert y alpha. La clase `AbstractImageLayer`

del repositorio confirma además que guarda `position`, `size`, `data`, `colorMode`, `imageName`, `alphaChannel` y expone exactamente esas acciones. ¹⁹

Para tu componente, recomiendo separar **asset library** de **instancias en canvas**:

- `assetsById`: una vez subida o importada la imagen, queda reusable.
- `ImageObject`: instancia posicionable/transformable que referencia `assetId`.
- `BitmapObject`: instancia raster editable píxel a píxel, generalmente derivada de `ImageObject` o creada directamente al pintar.

Ese flujo permite cubrir tanto “inserté un bitmap y lo muevo/redimensiono” como “tengo una capa raster que edito con pincel/goma”. En una primera iteración, basta con upload + insert + transform; icon packs, GIF como series y byte-array import pueden entrar en un segundo hito para acercarse más a Lopaka. ²⁰

Generación de código en tiempo real

Lopaka reconstruye el bloque generado **cada vez que cambia una layer**, y esa salida depende de cuatro cosas: plataforma, layers visibles no-overlay en orden de stack, título de la screen y code settings. También usa el nombre de la screen para derivar el wrapper function name, puede extraer variables desde propiedades marcadas, excluye layers ocultas y permite prefijo/sufijo editables fuera del bloque generado. En el panel, además, la selección de una layer debe poder resaltar y hacer scroll hasta su línea correspondiente. ²¹

La forma más limpia de replicar eso en React es un pipeline en dos fases:

1. **Documento visual → IR serializable.**
2. **IR serializable → texto de salida.**

Eso evita acoplar el renderer visual al formato final y permite varios targets: JSON, pseudocódigo, C/C++, YAML o incluso un DSL propio. Además, como React recomienda no guardar estado redundante calculable durante el render, el string final no debería ser parte mutable del reducer; debería producirse con selectores memoizados a partir del documento y de `codeSettings`. ²²

```
export interface DrawOp {
  id: string;
  sourceObjectId: string;
  kind: 'rect' | 'circle' | 'line' | 'polygon' | 'text' | 'bitmap';
  args: Record<string, unknown>;
}

export interface GeneratedCodeResult {
  ir: DrawOp[];
  text: string;
  lineMap: Record<string, { startLine: number; endLine: number }>;
}
```

```
export interface CodegenSettings {
  wrapperFunction: boolean;
  includeComments: boolean;
  declareImages: boolean;
  declareVariables: boolean;
  clearDisplay: boolean;
  prefix: string;
  suffix: string;
}
```

Un esquema JSON razonable para la IR sería:

```
{
  "screen": {
    "title": "Main Screen",
    "width": 128,
    "height": 64,
    "background": "#000000"
  },
  "ops": [
    {
      "id": "rect-1",
      "sourceObjectId": "rect-1",
      "kind": "rect",
      "args": {
        "x": 0,
        "y": 0,
        "w": 128,
        "h": 64,
        "fill": "#000000",
        "stroke": "#ffffff",
        "strokeWidth": 1
      }
    },
    {
      "id": "text-1",
      "sourceObjectId": "text-1",
      "kind": "text",
      "args": {
        "x": 18,
        "y": 34,
        "text": "hola mundo",
        "fontFamily": "Default",
        "fontSize": 12,
        "fill": "#ffffff"
      }
    }
  ]
}
```

```

    }
  ]
}

```

Y las actualizaciones incrementales pueden pensarse así, aunque el panel textual final se regenere entero en cada commit:

```

[
  {
    "type": "object/create",
    "objectId": "rect-1"
  },
  {
    "type": "code/ir-upsert",
    "sourceObjectId": "rect-1"
  },
  {
    "type": "object/create",
    "objectId": "text-1"
  },
  {
    "type": "code/ir-upsert",
    "sourceObjectId": "text-1"
  },
  {
    "type": "object/update",
    "objectId": "text-1",
    "changed": ["x", "y"]
  },
  {
    "type": "code/ir-upsert",
    "sourceObjectId": "text-1"
  }
]

```

Para el panel inferior, replicaría la idea de Lopaka casi literalmente:

- Zona `prefix` editable.
- Zona `generated` solo lectura.
- Zona `suffix` editable.
- `lineMap` por `sourceObjectId` para highlight sincronizado con la selección.
- Opción de wrapper function, comments, declare images, declare variables y clear display.

El hecho de que Lopaka ya derive comentarios y variables de los nombres de layer también sugiere una regla útil para tu diseño: **todo objeto debe tener `name` estable**, aunque el usuario nunca lo vea demasiado. Eso ayuda tanto en el inspector como en el código. ²³

Un ejemplo completo del **documento** para la composición final pedida —texto `hola mundo` y rectángulo negro con borde ocupando el canvas— podría ser este:

```
{
  "screen": {
    "title": "Screen 1",
    "width": 128,
    "height": 64,
    "background": "#000000"
  },
  "objectsById": {
    "rect-1": {
      "id": "rect-1",
      "kind": "rectangle",
      "name": "frame",
      "locked": false,
      "hidden": false,
      "zIndex": 0,
      "x": 0,
      "y": 0,
      "width": 128,
      "height": 64,
      "fill": "#000000",
      "stroke": "ffffff",
      "strokeWidth": 1
    },
    "text-1": {
      "id": "text-1",
      "kind": "text",
      "name": "label",
      "locked": false,
      "hidden": false,
      "zIndex": 1,
      "x": 18,
      "y": 34,
      "text": "hola mundo",
      "fontFamily": "Default",
      "fontSize": 12,
      "fill": "ffffff",
      "align": "left"
    }
  },
  "zOrder": ["rect-1", "text-1"],
  "assetsById": {}
}
```

Y una salida textual genérica derivada de ese documento podría verse así:

```
// [BEGIN generated]
drawRect(0, 0, 128, 64, { fill: "#000000", stroke: "ffffff", strokeWidth: 1 });
drawText(18, 34, "hola mundo", { fontFamily: "Default", fontSize: 12, fill:
"ffffff" });
// [END generated]
```

Accesibilidad, rendimiento, pruebas y roadmap

Accesibilidad

React recuerda que la accesibilidad sigue dependiendo en gran parte de HTML semántico, `aria-*`, focus management y soporte de teclado; además advierte que patrones basados solo en pointer/mouse suelen romper la experiencia de teclado. Para un editor como este, eso se traduce en: toolbar con `button` reales y `aria-pressed` para la herramienta activa; canvas con `tabIndex={0}`; shortcuts visibles cuando no hay selección; flechas para nudging; `Shift+flechas` para movimiento más grande; `Delete` / `Backspace` para borrar; `Escape` para salir de edición; y manejo explícito del foco cuando se entra en edición de texto o se abre el panel/modal de imágenes. También conviene evitar el patrón frágil de “click outside solamente” y soportar `onBlur`, `onFocus` y `Escape` como vías equivalentes. ²⁴

Rendimiento

El rendimiento depende menos del renderer elegido que de mantener el **modelo de estado pequeño y plano**, usar refs para interacción de alta frecuencia, y minimizar recomputaciones derivadas. Por eso recomiendo: documento normalizado; `hover`, `dragSession` y `pointer capture` en refs; commit de cambios de geometría en `requestAnimationFrame` o al menos agrupados por gesto; memoización del IR de código y de bbox derivados; canvas offscreen para thumbnails y bitmaps; y una lista de layers virtualizable si el documento crece mucho. Si se adopta Konva para parte del render, sus propias guías de rendimiento apuntan en la misma dirección: layer management, batch draw, shape caching y desactivar listeners en shapes estáticas. ²⁵

Estrategia de pruebas

La combinación más sana es:

- **Unit tests** para reducir, history, geometría, hit-testing y codegen.
- **Integration tests** con React Testing Library para asegurar que toolbar, canvas, inspector y code panel se comportan como lo haría un usuario.
- **Browser/component tests** con Playwright para drag real, contexto, doble click, layout, foco y visual regression.
- **Mocks** con Vitest para uploads, decodificación de assets y persistencia local.

Esta estrategia está muy alineada con lo que recomiendan las herramientas oficiales: Testing Library prioriza pruebas que se parezcan al uso real; Playwright corre en navegadores reales; y Vitest aporta las utilidades de mocking necesarias cuando el editor interactúa con servicios o APIs del navegador. ²⁶

Un ejemplo mínimo de unit test para history/reducer:

```
import { describe, it, expect } from 'vitest';
import { editorReducer, initialState } from './editorReducer';

describe('history', () => {
  it('agrupa un drag completo como una sola entrada de undo', () => {
    let state = initialState();

    state = editorReducer(state, {
      type: 'object/create',
      object: {
        id: 'rect-1',
        kind: 'rectangle',
        name: 'frame',
        locked: false,
        hidden: false,
        zIndex: 0,
        x: 0,
        y: 0,
        width: 10,
        height: 10,
        fill: '#000',
        stroke: '#fff',
        strokeWidth: 1
      }
    });

    state = editorReducer(state, { type: 'history/begin', label: 'move rect' });
    state = editorReducer(state, {
      type: 'object/update',
      id: 'rect-1',
      patch: { x: 40, y: 20 }
    });
    state = editorReducer(state, { type: 'history/commit' });

    expect(state.document.objectsById['rect-1']).toMatchObject({ x: 40, y:
20 });
    expect(state.history.past).toHaveLength(1);
  });
});
```

Un ejemplo de integration test para sincronía inspector → código:

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { LopakaLikeEditor } from './LopakaLikeEditor';

it('actualiza el código al cambiar el ancho de un rectángulo desde el
inspector', async () => {
  const user = userEvent.setup();
  render(<LopakaLikeEditor />);

  await user.click(screen.getByRole('button', { name: /rectangle/i }));
  await user.pointer([
    { target: screen.getByLabelText(/canvas/i), coords: { x: 10, y: 10 }, keys:
    '[MouseLeft>]' },
    { target: screen.getByLabelText(/canvas/i), coords: { x: 90, y: 50 } },
    { target: screen.getByLabelText(/canvas/i), keys: '/MouseLeft]' }
  ]);

  await user.clear(screen.getByLabelText(/w$/i));
  await user.type(screen.getByLabelText(/w$/i), '120');

  expect(screen.getByLabelText(/generated code/i)).toHaveTextContent('120');
});
```

Y un ejemplo de browser test para interacciones que no vale la pena simular en JSDOM:

```
import { test, expect } from '@playwright/test';

test('paint usa clic derecho para borrar', async ({ page }) => {
  await page.goto('/editor-demo');
  await page.getByRole('button', { name: /paint/i }).click();

  const canvas = page.getByLabel('canvas');

  await canvas.click({ position: { x: 20, y: 20 } }); // pinta
  await canvas.click({ position: { x: 20, y: 20 }, button: 'right' }); // borra

  await expect(page.getByLabel('generated code')).toContainText('bitmap');
});
```

Roadmap de implementación

Hito	Entregable	Esfuerzo
Base documental y modelo	EditorDocument, unions TS, reducer, selección, z-order, assets	Medio

Hito	Entregable	Esfuerzo
Stage y viewport	SVG/canvas híbrido, pan/zoom, coordenadas, marquee, hit-testing base	Medio
Herramientas núcleo	<code>select</code> , <code>rectangle</code> , <code>circle</code> , <code>line</code> , <code>text</code>	Medio
Inspector y código	panel de propiedades, lineMap, codegen IR + panel en vivo	Medio
Bitmaps e imágenes	upload, insert, resize, alpha, paint/erase, transformaciones raster	Alto
Geometría avanzada	<code>triangle</code> , <code>polygon</code> , vertex mode, clic derecho remove vertex, double click	Alto
History y teclado	batching por gesto, undo/redo, Esc/Delete/arrows, shortcuts help	Medio
Hardening	accesibilidad, performance, virtualización, RTL/Playwright, pulido API	Medio

flowchart LR

```

A[Modelo de documento y reducer] --> B[Viewport e interacción base]
B --> C[Rect Circle Line Text]
C --> D[Inspector y codegen]
D --> E[Imágenes y bitmap paint]
E --> F[Triangle y Polygon vertex mode]
F --> G[Undo Redo y shortcuts]
G --> H[Accesibilidad pruebas y optimización]

```

La decisión arquitectónica que más valor aporta es esta: **un solo componente público, modelo de documento propio, reducer con history por comandos, renderer híbrido y generación de código en dos fases**. Esa combinación replica bien la funcionalidad documentada de Lopaka, respeta la restricción de una sola screen, mantiene la UI editable con precisión numérica, y evita que la elección del renderer dicte el modelo del producto. ²⁷

¹ ³ ⁴ ⁵ ¹⁴ ¹⁵ ¹⁶ ¹⁸ ²⁷ <https://lopaka.app/pages/docs/editor-overview>

<https://lopaka.app/pages/docs/editor-overview>

² ¹² <https://www.w3.org/TR/SVG2/>

<https://www.w3.org/TR/SVG2/>

⁶ <https://react.dev/>

<https://react.dev/>

⁷ ²⁵ [Choosing the State Structure – React](#)

<https://react.dev/learn/choosing-the-state-structure>

⁸ <https://www.typescriptlang.org/docs/handbook/2/objects.html>

<https://www.typescriptlang.org/docs/handbook/2/objects.html>

9 <https://raw.githubusercontent.com/sbrin/lopaka/refs/heads/main/src/core/history.ts>
<https://raw.githubusercontent.com/sbrin/lopaka/refs/heads/main/src/core/history.ts>

10 <https://www.w3.org/TR/pointerevents/>
<https://www.w3.org/TR/pointerevents/>

11 <https://react.dev/learn/manipulating-the-dom-with-refs>
<https://react.dev/learn/manipulating-the-dom-with-refs>

13 <https://konvajs.org/docs/index.html>
<https://konvajs.org/docs/index.html>

17 https://konvajs.org/docs/events/Custom_Hit_Region.html
https://konvajs.org/docs/events/Custom_Hit_Region.html

19 20 <https://lopaka.app/pages/docs/importing-images>
<https://lopaka.app/pages/docs/importing-images>

21 22 23 <https://lopaka.app/pages/docs/code-generation>
<https://lopaka.app/pages/docs/code-generation>

24 <https://legacy.reactjs.org/docs/accessibility.html>
<https://legacy.reactjs.org/docs/accessibility.html>

26 <https://testing-library.com/docs/react-testing-library/intro/>
<https://testing-library.com/docs/react-testing-library/intro/>